**Adaptive CPU Scheduling Using Reinforcement Learning for Dynamic Workloads**

Review 3: Complete Work and Final Paper Submission

Lab Assessment 10

[Student Name(s)] [Roll Number(s)] [Department Name] [Institution Name] [Academic Year]

Date of Submission: [Date]

## Table of Contents

[Auto-generate Table of Contents in Word: References > Table of Contents]

## 1. Project Title

Adaptive CPU Scheduling Using Reinforcement Learning for Dynamic Workloads

## 2. Abstract

CPU scheduling in modern operating systems must accommodate workloads whose characteristics change over time. Traditional scheduling policies such as First-Come First-Served (FCFS), Shortest Job First (SJF), Round Robin, and Priority Scheduling each perform well under specific workload conditions but degrade when those conditions shift. This project presents a safety-aware self-healing CPU scheduling simulator that detects performance degradation during execution, diagnoses a likely cause from observable evidence, selects a recovery action using either a predeclared rule or a tabular Q-learning agent, verifies the outcome over a bounded observation window, and then accepts, rolls back, or falls back to a safe configuration.

The system is implemented as a discrete-event simulator in Python with four classical scheduling policies, a causal monitoring and degradation detection pipeline, a tri-state verifier, and a comprehensive incident ledger. The evaluation uses seven synthetic workload families with 210 held-out test traces per arm, evaluated against ten predeclared paired comparisons using family-stratified crossed bootstrap with 1000 replicates. A counterfactual replay audit measures the true effect of each intervention by re-running the same controller on the same workload with the intervention forced to hold.

The principal finding is that both the waiting-time rule and the learned Q-learning controller reduce mean waiting time by approximately 8.5 units over the FCFS baseline (from 130.84 to 122.39 and 122.56 respectively, with 95% confidence intervals excluding

zero), but neither approaches the fixed SJF oracle at 94.66. The Q-learning arm is statistically indistinguishable from the hand-written rule (difference of 0.18 units, 95% CI [-0.04, +0.48]). Every accepted intervention across all arms was genuinely helpful (65 of 65), but the verifier can certify only 8.0% of beneficial interventions due to the limited statistical power of a single observation window. The major limitation is that this is a simulation study, not a kernel modification, and no reinforcement-learning advantage over the rule-based approach is demonstrated.

## 3. Keywords

CPU scheduling, adaptive scheduling, reinforcement learning, Q-learning, workload simulation, self-healing systems, degradation detection, verification and rollback, performance monitoring, policy selection

## 4. Introduction

Process scheduling is a fundamental responsibility of any operating system kernel. The scheduler determines which process runs next on the CPU, directly affecting system throughput, response time, and fairness to individual processes. Classical scheduling algorithms—FCFS, SJF, Round Robin, and Priority—each encode a fixed policy that performs optimally under specific workload conditions but may behave poorly when those conditions change (Silberschatz et al., 2018).

In practice, workloads are rarely stationary. A system that begins processing a batch of long-running compute jobs may suddenly receive a burst of interactive requests. The policy optimal for the initial batch may be counterproductive for the interactive burst. This observation motivates adaptive scheduling: a system that monitors its own performance, recognizes degradation, and changes its scheduling policy in response.

Reinforcement learning (RL) offers a natural framework for adaptive decision-making under uncertainty. An RL agent observes the system state, selects an action, receives a reward signal, and learns a policy that maximizes cumulative reward over time (Sutton & Barto, 2018). Several researchers have applied RL to scheduling problems in cloud computing, job shops, and real-time systems (Zhou et al., 2024; Kaveri & Lahande, 2023). However, most existing approaches focus on finding better scheduling policies without addressing the safety of adaptation itself: what happens when the agent makes an incorrect decision?

This project investigates a different question: can an adaptive scheduler recognize when it is degrading, recover safely from workload shifts, and avoid harmful or unnecessary interventions under uncertainty? The implemented system is a safety-aware self-healing CPU scheduling simulator. It does not claim that Q-learning produces a better scheduler

than existing methods. Instead, it investigates whether a recovery loop with explicit verification and rollback can provide safety guarantees that existing adaptive approaches lack.

The scope of this project is deliberately limited to a discrete-event simulation. There is no kernel module, no syscall, no real hardware measurement, and no real workload trace. All workloads are synthetically generated from declared families with reproducible seeds. The evaluation uses paired bootstrap comparisons on identical workloads, counterfactual replay audits, and derived thresholds from the no-action control. The contribution is an integrated prototype that combines adaptive policy selection with verification, rollback, and a comprehensive incident ledger that makes every decision traceable to its evidence.

## 5. Related Work / Literature Survey

This section reviews research published between 2021 and 2026 across several themes relevant to this project: reinforcement learning for scheduling, adaptive and workload-aware systems, safe RL and verification, self-healing systems, and performance monitoring. At least 30 distinct papers are reviewed, synthesised by theme rather than listed as disconnected summaries.

### 5.1 Reinforcement Learning for Resource Scheduling

The application of deep reinforcement learning (DRL) to resource scheduling in cloud computing has attracted significant attention. Zhou et al. (2024) provide a comprehensive survey of DRL methods for cloud resource scheduling, covering task scheduling, workflow management, and resource provisioning. They identify that DRL agents can learn scheduling policies that outperform static heuristics in dynamic environments, but note challenges including reproducibility, generalisation to unseen workloads, and the gap between simulation and deployment. Similarly, Kaveri and Lahande (2023) apply reinforcement learning to improve resource scheduling and load balancing in cloud environments, using simulation-based experiments with WorkflowSim to demonstrate that RL agents can learn to distribute tasks more effectively than round-robin or random allocation.

Lin et al. (2022) propose a scheduling algorithm based on proximal policy optimization (PPO) for heterogeneous environments, combining graph neural networks for encoding application structure with RL for dispatch decisions. Their evaluation reports a 13% reduction in average completion time compared to existing heuristics, though the method requires a trained neural network and is evaluated on synthetic DAGs rather than real system traces. Paduraru et al. (2023) investigate task scheduling using DQN, Double-DQN, and Dueling-DQN, comparing these against genetic algorithms in job-shop scenarios,

finding that RL methods can match or exceed evolutionary approaches on certain metrics.

In the domain of high-performance computing batch scheduling, Kolker-Hicks et al. (2023) propose a reinforcement-learning-based backfilling strategy that achieves up to 17x better scheduling performance compared to EASY backfilling. The authors use a tabular Q-learning approach similar in spirit to the one used in this project, though applied to a different domain (HPC job backfilling rather than CPU scheduling). Hayat et al. (2023) present a machine-learning-based task scheduler for heterogeneous systems that uses learned predictors to estimate execution times and select appropriate processing elements.

Zhou and Lin (2023) develop a CPU frequency scheduling method using temporal encoding-based deep reinforcement learning for real-time applications on embedded devices. Their approach derives an effective DVFS governor that accounts for workload/system patterns challenging for Linux's built-in solutions, demonstrating 3-11% energy savings on MiBench workloads. Ziaei and Ranjbar (2023) present a reinforcement learning algorithm for scheduling parallel processors, addressing the problem of assigning tasks to identical-speedup processors to minimize makespan. Ismail, Khalifa, and El-Khoribi (2025) survey DRL-based resource scheduling approaches for multi-access edge computing, covering content caching, computation offloading, and resource management across 97 papers from 2019-2023. Kuang et al. (2023) propose ODRL, a deep reinforcement learning framework for priority scheduling in hybrid cloud environments, combining AHP-TOPSIS workload characterization with prioritized experience replay to reduce datacenter running costs.

## 5.2 Adaptive and Workload-Aware Scheduling

Adaptive scheduling systems that respond to changing workload characteristics have been studied across multiple computing domains. Sabek et al. (2022) present LSched, a workload-aware learned query scheduler for analytical database systems that uses reinforcement learning to minimize both average and tail latency by learning which queries and operators to schedule under different workload conditions. Their approach shares the key insight of this project—that scheduling decisions should be conditioned on observed workload characteristics rather than fixed rules—though applied to database query scheduling rather than CPU dispatch.

McClure et al. (2022) investigate efficient scheduling policies for microsecond-scale tasks, challenging the tradeoff between low latency and high utilization. Sung et al. (2023) propose a decentralized application-level adaptive scheduling mechanism for multi-instance DNNs on mobile devices, using deep reinforcement learning to converge to Nash equilibrium scheduling. Fu et al. (2024) present ALPS (Adaptive Learning, Priority OS Scheduler), an application-aware kernel scheduler for serverless functions that

approximates Shortest Remaining Processing Time while preventing starvation of longer functions through learned priority adjustments (Fu, Cui, Yang, Qi, & Qiu, 2024). Their work is the closest real-system analogue to the adaptive scheduling question this project addresses, though ALPS operates inside the kernel rather than on a separate controller and does not implement explicit outcome verification.

The IADA architecture (Meyer, da Silva, Kirchoff, & De Rose, 2022) presents a dynamic interference-aware cloud scheduling system that uses machine learning for workload classification and Bayesian changepoint detection for online inference of workload variations. This shares the detection and response paradigm of the current project, though at the cloud infrastructure level rather than CPU scheduling. The dynamic adaptive scheduling framework of Wang et al. (2023) combines fast, low-overhead scheduling with sophisticated, high-performance scheduling, using a runtime classifier to select the better scheduler based on workload characteristics.

### 5.3 Safe Reinforcement Learning and Verification

The safety of reinforcement learning agents deployed in real systems has become a major research concern. Gu et al. (2024) provide a comprehensive review of safe RL methods, theories, and applications in IEEE Transactions on Pattern Analysis and Machine Intelligence, covering constraint-based approaches, shielding mechanisms, and verification methods. They identify that most safe RL research addresses training-time safety (preventing the agent from exploring dangerous actions during learning) but less attention has been paid to deployment-time safety (verifying that the learned policy will not cause harm when deployed).

Hao et al. (2023) survey state-wise safe reinforcement learning, categorizing approaches into hierarchical agents, action projection, and Lyapunov-based methods. Their survey identifies the fundamental tension between safety constraints and exploration: an agent that cannot explore unsafe actions may never learn the optimal policy. Wei et al. (2024) address safe RL with instantaneous hard constraints, proposing LSVI-AE which achieves sublinear regret while maintaining hard constraint satisfaction. Brunke et al. (2022) investigate safe reinforcement learning in the context of real-world robotics, noting that sim-to-real transfer of safety guarantees remains an open challenge.

Pekeric et al. (2023) provide a systematic review of security and safety in self-adaptive systems, identifying that self-adaptation processes must themselves be secured against adversarial manipulation. Their work is relevant because the current project's self-healing loop is a form of self-adaptation whose decisions could be exploited if the monitoring statistics were manipulable. Wachi et al. (2024) survey constraint formulations in safe RL, providing a taxonomy of how safety constraints can be specified and enforced.

## 5.4 Self-Healing and Self-Adaptive Systems

The concept of self-healing—systems that autonomously detect and recover from degradation—has been studied across multiple domains. Hafaiedh and Slimane (2022) propose a distributed formal model for self-healing behaviours in autonomous systems, covering failure detection through self-recovery. Their formal model provides specification and verification frameworks that are conceptually similar to the verification mechanism in the current project, though applied to different classes of autonomous systems.

Kumar (2023) investigates AI-based approaches for fault detection and recovery in self-healing networks, demonstrating that machine learning can proactively detect anomalies and trigger recovery actions. The detect-diagnose-act-verify pipeline pattern in the current project is a recurring architectural motif in self-healing systems across multiple domains, from autonomous fault-tolerant systems (Hafaiedh & Slimane, 2022) to cloud service recovery.

Yahmed et al. (2023) present an intentional forgetting-driven self-healing method for deep reinforcement learning systems, addressing the problem of RL agents whose learned policies degrade due to distribution shift. Their approach is complementary to the current project: where the current project rolls back individual harmful actions, their method resets the learned policy itself. Rauba, Seedat, Kacprzyk, and van der Schaar (2024) present self-healing machine learning (SHML), a framework that autonomously diagnoses the reason for degradation and proposes diagnosis-based corrective actions, formalizing self-healing as an optimization problem over adaptation actions to minimize expected risk under a shifted data-generating process.

## 5.5 Performance Monitoring and Degradation Detection

Effective adaptive systems require reliable performance monitoring and degradation detection. Kang et al. (2024) analyse reliability for systems with self-healing mechanisms in degradation-shock dependence processes, modelling the interaction between gradual degradation and sudden shocks—directly relevant to the current project's burst_shock and arrival_storm drift mechanisms.

The integration of graceful degradation and recovery through requirement-driven adaptation is studied by researchers at ACM (2024), who treat degradation and recovery as complementary adaptation tasks: degradation temporarily weakens system requirements, and recovery restores them when conditions improve. This framing parallels the current project's approach of accepting, rejecting, or leaving interventions inconclusive based on observed evidence.

## 5.6 Scheduling Evaluation and Experimental Methods

The methodology for evaluating scheduling algorithms has evolved significantly. Karimi-Mamaghan et al. (2022) integrate Q-learning into meta-heuristic algorithms for the permutation flowshop scheduling problem, demonstrating that learned operator selection can improve solution quality. Yang et al. (2024) survey advancements in Q-learning meta-heuristic optimization, providing context for how tabular Q-learning—as used in this project—relates to more complex variants.

Zhu et al. (2023) investigate intelligent scheduling using advanced Q-learning and SARSA models for job-shop optimization, published in Electronics (MDPI). Their comparison of Q-learning and SARSA is relevant because the current project uses Q-learning with epsilon-greedy exploration; their work provides empirical context for expected convergence behaviour. Chen, Chen, Li, Liang, and Li (2023) propose DRJOA, a deep reinforcement learning framework for intelligent resource management and job scheduling optimization in edge computing, demonstrating improved resource utilization compared to static scheduling baselines through a priority-aware scheduling algorithm with prioritized experience replay.

The use of counterfactual analysis in evaluating intervention effectiveness has been applied in scheduling contexts by several researchers. The approach of re-running the same controller on the same workload with one action modified—as implemented in the current project's matched replay audit—is a standard causal inference technique adapted to the scheduling domain.

### 5.7 Literature Summary

## 6. Inferences and Identified Research Gaps

From the reviewed literature, several inferences can be drawn. First, reinforcement learning has been successfully applied to scheduling problems across multiple domains—cloud computing, database queries, HPC batch jobs, and embedded systems—but most approaches focus on finding better static policies rather than on the safety of adapting between policies at runtime (Zhou et al., 2024; Kaveri & Lahande, 2023). Second, the safe RL literature identifies the tension between exploration and safety but primarily addresses training-time constraints; deployment-time safety for scheduling remains underexplored (Gu et al., 2024; Hao et al., 2023).

Third, self-healing systems in networking and manufacturing demonstrate that autonomous detection and recovery is feasible (Hafaiedh & Slimane, 2022; Kumar, 2023), but few implementations combine detection with explicit verification of recovery effectiveness. Fourth, the literature on workload-aware scheduling (Sabek et al., 2022; Meyer et al., 2022) shows that adapting to workload characteristics improves performance, but these systems

typically do not provide mechanisms to roll back incorrect adaptations.

The identified gap is the integration of adaptive policy selection with explicit outcome verification and rollback in the context of CPU scheduling. Existing adaptive schedulers commit to policy changes without verifying their effect; existing safe RL methods address constraints but not the question of whether a specific adaptation improved performance. This project addresses this gap by implementing a self-healing loop that detects degradation, applies a recovery action, verifies the outcome against a phase-adjusted baseline, and rolls back interventions that cannot be certified as helpful.

## 7. Problem Statement

The technical problem addressed by this project is two-fold:

Problem 1 — Scheduling Policy Comparison: Given a set of classical CPU scheduling policies (FCFS, SJF, Round Robin, Priority) and a collection of synthetic workload instances drawn from declared families, determine which policy minimizes mean waiting time on each workload, and quantify the uncertainty of the comparison using paired bootstrap confidence intervals.

Problem 2 — Safe Adaptive Recovery: Given a running schedule whose performance is degrading relative to its own established baseline, detect the degradation, select a recovery action (policy change or quantum adjustment), verify whether the action improved performance, and roll back the action if it cannot be verified as beneficial. The system must avoid harmful interventions and must not trigger the safe fallback (pinning a single configuration) due to inconclusive evidence alone.

Assumptions: The workload is a sequence of processes with known arrival times, burst times, and priorities, generated synthetically from declared families. The scheduler operates on a single CPU with a FIFO ready queue. Context switches incur a fixed cost. The monitoring and detection pipeline sees only causal information (arrived processes, completed processes, and past decisions).

Scope: This is a discrete-event simulation study. It does not model a real kernel, real workload traces, I/O, multiprocessor effects, or cache behaviour. The measurable outcomes are mean waiting time, turnaround time, response time, CPU utilization, throughput, and context-switch count, plus healing-specific outcomes (incidents, interventions, acceptance rates, rollback rates, and counterfactual audit gains).

## 8. Objectives and Novelty of the Proposed Work

The project has the following measurable objectives:

1. Implement a discrete-event CPU scheduling simulator supporting four classical policies (FCFS, SJF, Round Robin, Priority) with configurable context-switch costs and quantum sizes.

2. Implement a self-healing controller that monitors scheduling performance, detects degradation events, diagnoses likely causes, and selects recovery actions.

3. Implement a tabular Q-learning recovery agent with epsilon-greedy exploration, masked actions, and forced first visits, trained on synthetic workload families.

4. Implement a tri-state verifier (accepted, rejected, inconclusive) with phase-adjusted residuals derived from the no-action control distribution.

5. Evaluate the system on held-out test workloads using paired bootstrap comparisons and counterfactual replay audits.

6. Build an interactive web dashboard for classroom demonstration and a comprehensive test suite ensuring reproducibility and correctness.

Novelty: The contribution is not a novel RL algorithm. Tabular Q-learning is a well-established technique (Watkins & Dayan, 1992). The novelty lies in the integration of: (a) adaptive policy selection with explicit outcome verification, (b) a tri-state verdict system that distinguishes confirmed failure from insufficient evidence, (c) a counterfactual replay audit that provides causal evidence for each intervention's effect, and (d) a comprehensive incident ledger that records every decision with its evidence, cost, and outcome. This integration is an implemented prototype that demonstrates a safety-first approach to adaptive scheduling, where the cost of incorrect adaptation is bounded by verification and rollback.

## 9. Proposed Methodology

The methodology follows a structured experimental pipeline from workload generation through scheduling, monitoring, adaptation, verification, and evaluation.

### 9.1 Workload Generation

Seven workload families are defined, each specifying arrival patterns (uniform, Poisson, staggered), burst distributions (uniform, bimodal, staggered-interactive), and priority patterns (uniform, high-priority-skewed). Each workload instance is generated from a (family, seed) pair using numpy.random.default_rng, ensuring reproducibility. A content fingerprint proves that two arms evaluated the same instance. Optional drift injection (burst_shock, arrival_storm, phase_transition, priority_inversion) modifies the workload at a declared fraction of the arrival span while preserving the pre-injection prefix.

## 9.2 Scheduling Engine

The RuntimeSimulator implements a discrete-event loop with one CPU and a FIFO ready queue. At each decision epoch, it builds a RuntimeObservation (arrived, unfinished ready set plus counters) and passes it to the controller. FCFS and RR dispatch the head of the queue; SJF picks the shortest remaining burst; Priority picks the highest priority. Non-RR service is non-preemptive; RR service is min(quantum, remaining_burst). Switching cost is charged when the running process changes.

## 9.3 Monitoring and Detection

The CausalMonitor maintains a rolling window of the most recent 8 completed processes' waiting times, with an EWMA reference (alpha=0.35) and dispersion band (alpha=0.25). The DegradationDetector declares a degradation event when the window mean exceeds the reference plus 1.25 times the band, for 2 consecutive evaluations, outside a cooldown of 3 decisions, with sufficient ready processes and visible work, and above the congestion gate (min_load_factor=1.0). Every suppression path has its own reason, logged in the incident ledger.

## 9.4 Diagnosis and Recovery Action Selection

Upon detection, the diagnosis module maps visible ready-set evidence to a primary cause (head-of-line blocking, fairness starvation, priority-ordering anomaly, tail dispersion, queue congestion, or unexplained). The recovery policy then selects from 8 actions: set_fcfs, set_sjf, set_priority, set_round_robin, grow_quantum, shrink_quantum, hold, revert_to_last_accepted. The policy is either a predeclared diagnosis-to-action rule or a tabular Q-learning agent with epsilon-greedy exploration, masked unavailable actions, and forced first visits.

## 9.5 Verification and Outcome

After applying an action, a verification window of 5 decisions and at least 3 completions observes the outcome. The verifier computes a phase-adjusted closure residual by predicting the no-action post-window queue level from trigger-time observables (trigger_load, ready_count, log1p(visible_work)) using a least-squares model fitted on the training control. The residual is compared with ±0.30 boundaries (95th and 5th percentiles of the control residual). Outcomes: accepted (keep the change), rejected (rollback, failure counter +1), inconclusive (rollback, no penalty, no failure count). Two consecutive confirmed failures trigger the safe fallback.

## 9.6 Experimental Design

Training: 5 seeds (7101-7105), 1200 traces each, 6 families (all except poisson_arrivals). Validation: seed 8201, 140 traces. Held-out test: seed 23819, 210 traces. All thresholds were derived from the training control before test evaluation. Ten predeclared paired comparisons use family-stratified crossed bootstrap (1000 replicates, seed 104729). Counterfactual audit replays every intervention with forced holds on the same controller and workload.

**Figure 1. Experimental methodology pipeline**

*Figure 1. Experimental methodology pipeline*

## 10. Overall Conceptual and Architectural Diagram

Figure 2 presents the overall system architecture. The pipeline flows from workload generation (with optional drift injection and burst estimation) through the discrete-event scheduler, which interacts with the self-healing controller. The controller's loop comprises monitoring, detection, diagnosis, recovery policy selection, action application, verification, and rollback/fallback. The metrics and evaluation modules consume the schedule results and incident ledger to produce the final evaluation.

**Figure 2. Overall system architecture of the adaptive CPU scheduling pipeline**

*Figure 2. Overall system architecture of the adaptive CPU scheduling pipeline*

## 11. Description of Individual Modules

## 12. Module-wise Workflow Diagrams

### 12.1 Workload Generation and Scheduling Workflow

Figure 4 shows the workflow from workload family and seed selection through generation, optional drift injection, and execution by the RuntimeSimulator. The generator produces fingerprinted Workload objects; the simulator executes them under a controller, producing ScheduleResult with per-process outcomes.

**Figure 3. Self-healing controller workflow (one trace)**

*Figure 4. Workload generation and scheduling workflow*

### 12.2 Degradation Detection and Diagnosis Workflow

Figure 5 details the detection pipeline. Each completion notice updates the rolling EWMA reference and band. When the window mean exceeds the threshold, hysteresis is checked, followed by the work gate, congestion gate, and cooldown. Only when all gates pass is an incident declared, and the ready-set evidence is used to diagnose a primary cause.

**Figure 4. Workload generation and scheduling workflow**

*Figure 5. Degradation detection and diagnosis workflow*

### 12.3 Verification and Acceptance Workflow

Figure 6 shows the tri-state verification. After an action is applied, the verification window collects completions and computes the phase-adjusted closure residual. If the residual exceeds +0.30, the change is accepted; if below -0.30, it is rejected (confirmed failure); otherwise it is inconclusive and rolled back without penalty.

**Figure 5. Degradation detection and diagnosis workflow**

*Figure 6. Verification, acceptance, and rollback workflow*

### 12.4 Self-Healing Loop Workflow

Figure 3 presents the complete self-healing controller workflow for one trace, showing the cycle from workload generation through simulation, monitoring, detection, diagnosis, action selection, verification, and the three possible outcomes (accepted, rejected, inconclusive), with the safe fallback after two consecutive failures.

**Figure 6. Verification, acceptance, and rollback workflow**

*Figure 3. Self-healing controller workflow (one trace)*

### 12.5 Frontend-to-Backend Execution Flow

Figure 7 shows how the interactive web console invokes the simulation engine. The browser sends an HTTP request to the FastAPI server, which validates the request, calls the simulation engine, validates the output model, and returns JSON with inline SVG charts.

**Figure 7. Frontend-to-backend simulation execution flow**

*Figure 7. Frontend-to-backend simulation execution flow*

## 13. Dataset Description

This project uses synthetically generated workloads rather than a public benchmark dataset. This is a deliberate design choice: real workload traces are not available for CPU scheduling at the process level, and the project's focus is on the adaptive mechanism rather than on specific real-world workload characteristics.

**Data source: Synthetic generation via declared families and seeds.**

**Generation method:**

Seven workload families, each defining arrival patterns (uniform over [0, arrival_window], Poisson with declared rate, or staggered with uniform gaps), burst distributions (uniform over [burst_min, burst_max], bimodal with short/long modes, or staggered-interactive with one long process and short followers), and priority patterns (uniform or high-priority-skewed). Each instance is drawn using numpy.random.default_rng(seed), making (family, seed) a reproducible identifier.

**Dataset size:**

Training: 5 seeds (7101-7105), 1200 episodes per seed cycling through 6 families (all except poisson_arrivals) = 6000 total training episodes. Validation: seed 8201, 7 families x 20 repetitions = 140 traces. Held-out test: seed 23819, 7 families x 30 repetitions = 210 traces. Each trace contains 15 processes with arrival times, burst times, and priorities.

**Random seed strategy:**

Training seeds 7101-7105 are independent replicates. Validation seed 8201 and test seed 23819 were declared in the configuration before the frozen run. The derivation seed (104729) for bootstrap comparisons is separate. Child seeds are derived via SHA-256 hashing of the master seed and position identifiers (config.derive_seed), preventing random stream correlation between different draws.

Units: All time quantities (arrival, burst, waiting, turnaround) are in abstract simulated time units.

## 14. Experimental Setup and Implementation Details

**Software environment:**

Python 3.11.2 (CPython), NumPy 2.4.6, pandas 3.0.6, Matplotlib 3.11.2, pytest 9.1.1. The frozen study requires approximately 47 seconds wall-clock time on a standard desktop.

**Hardware:**

The study was recorded on a desktop system (specific hardware not documented in the repository). The pipeline is deterministic and byte-reproducible across independent runs.

**Reproducible commands:**

# Activate environment source .venv/bin/activate # Install dependencies pip install -r requirements.lock # Run the frozen healing study python main.py healing-study # Run the demo python main.py demo # Launch the live dashboard python tools/serve_dashboard.py # Open http://127.0.0.1:8000/ # Run tests pytest tests/

**Key parameters:**

**Output artifacts:**

The frozen study produces 21 machine-readable artifacts in results/healing/ (CSV tables, JSON summaries, incident ledgers, coverage data) plus 3 figures. The pipeline writes into a temporary directory per run; committed artifacts are the frozen run's output.

## 15. Evaluation Metrics

The following metrics are computed by evaluation/metrics.py from the finished ScheduleResult.

Additional reliability metrics from evaluation/reliability.py include P95 waiting time, max waiting time, starvation count (processes with waiting time > 3x median), Jain's fairness index over waiting times, recovery delay (median decisions between detection and verification close), false-intervention rate (from counterfactual audit), rollback rate, regression rate, and fallback rate.

## 16. Results and Discussion

### 16.1 Held-out Comparison (Test Seed 23819)

Table 3 presents the mean performance of each arm over 210 held-out test traces. The control (detector_only) is exactly FCFS, confirming that monitoring and detection alone do not alter the schedule when the policy never intervenes.

The ordering fixed_sjf < rule_wait ~ q_recovery < fixed_fcfs < rule_fairness is resolved at this design. The learned arm is not distinguishable from the waiting-time rule. The fairness rule is genuinely harmful on these families (136.78 vs 130.84 for FCFS), mapping starvation to Round Robin buys responsiveness at the cost of waiting time. Throughput and utilization are essentially identical across FCFS/SJF/priority arms because the same total work is completed; the differences are in who waits, not how much work is done.

### 16.2 Paired Comparisons with Uncertainty

Family-stratified crossed bootstrap over identical workloads, 1000 replicates, seed 104729.

The key comparisons: rule_wait vs detector_only yields -8.45 wait units (CI [-9.44, -7.35]), excluding zero. q_recovery vs fixed_fcfs yields -8.27 (CI [-9.31, -7.20]). The learned arm vs the rule yields +0.18 (CI [-0.04, +0.48]), which includes zero, confirming that the Q-learning controller is not statistically distinguishable from the hand-written rule.

### 16.3 Counterfactual Replay Audit

The counterfactual audit measures the true causal effect of each intervention by re-running the same controller on the same workload with the intervention forced to hold.

The waiting-time rule's interventions reduce total waiting by approximately 189 units on average and help 94.3% of the time. The learned controller's interventions are indistinguishable (+192.07, 95.0%). The fairness rule is genuinely harmful (76.7% false recovery rate). The SJF-start arm has almost nothing to fix (4 interventions, all harmful).

### 16.4 Verification Behaviour

This is the honest core of the study. The verifier accepts a population that is purely beneficial: 65 of 65 accepted interventions helped (mean gain +184.9). But it can certify only 8.0% of the genuinely helpful interventions (65 of 812); 91-92% end inconclusive and are rolled back. The reason is statistical: the best action moves the monitored statistic by 0.73 standard deviations of the residual, and approximately 15 windows per action would be needed for 80% power at alpha=0.05.

### 16.5 Discussion of Negative and Inconclusive Results

Several findings require careful discussion rather than celebration:

1. The loop does not beat fixed SJF. SJF achieves 94.66 mean waiting time; the best adaptive arms achieve 122.39-122.56. SJF is an oracle (it knows burst times) and cannot be beaten by any causal policy that does not also know burst times.

2. The Q-learning controller provides no advantage over the hand-written rule. The paired difference is 0.18 units with CI [-0.04, +0.48], which includes zero. The Q-learning agent is undertrained (1200 traces visit 29-33 of 648 states) and the coverage curve is flat after approximately 900 episodes.

3. The verifier is a safety filter, not a benefit estimator. It correctly certifies the extreme tail of helpful interventions but leaves the majority as inconclusive. This is by design: the system never keeps a configuration it cannot verify.

4. The fairness rule is empirically wrong on these families. Mapping fairness starvation to Round Robin costs 130.8 wait units per intervention. It is kept as an honest negative result, not as a recommended method.

## 17. Comparison with Existing Methods

Table 8 compares the implemented methods with relevant literature across several dimensions. Numerical comparisons across studies with different workloads, hardware, and assumptions are not meaningful; the comparison focuses on methodological characteristics.

## 18. Limitations of the Proposed Work

**Simulation vs. real kernel scheduling:** The system drives a discrete-event simulator, not a kernel. There is no kernel module, no syscall, no CFS/EEVDF integration, no hardware measurement. All quantities are simulator values.

**Synthetic workloads and external validity:** Every workload is generated from declared families with synthetic seeds. No real workload traces are used. The results may not generalise to production workloads with different arrival patterns, burst distributions, or resource contention.

**Burst times are visible by default:** SJF receives exact burst lengths, making it an oracle baseline. The burst estimator module adds modelled noise but is not calibrated against real traces. In the estimator-enabled experiment, SJF still receives exact values and acts as an oracle.

**No reinforcement-learning advantage demonstrated:** The Q-learning controller is statistically indistinguishable from the hand-written rule. The agent visits only 29-33 of 648 states and the coverage curve is flat after 900 episodes. No RL advantage is claimed.

**Verifier can certify only an extreme tail:** Only 8.0% of genuinely helpful interventions clear the acceptance boundary. The verifier is a safety filter that correctly identifies the best interventions but leaves the majority inconclusive.

**Detector false-trigger rate:** The control declares 0.62-0.65 events per trace without any intervention. Recall on injected degradations is 75.0% (burst shock) and 56.4% (arrival storm).

**Single-CPU, no I/O, fixed switch cost:** The simulator models one CPU with FIFO ready queue, no I/O, no page faults, no interrupts, no multicore, no cache effects. Switch cost is constant at 1 time unit.

**Limited state-action coverage:** The 648-state space is larger than the event supply of the study. Only 3.8-7.2% of reachable state-action pairs are visited during training.

**Threshold derivation on training control:** All thresholds (congestion gate, acceptance boundaries, phase model) are derived from the no-action control of the training split. The declared setting was chosen on training, not on validation numbers.

**Statistical power and uncertainty:** Bootstrap confidence intervals are conditional on the declared design (5 training seeds, 7 families, fixed repetitions). They are not claims about all workloads.

## 19. Conclusion and Future Scope

This project implements a safety-aware self-healing CPU scheduling simulator that detects performance degradation under changing workloads, adapts using learned or rule-based recovery policies, verifies the effects of its decisions, and rolls back harmful interventions under uncertainty. The system comprises a discrete-event simulator with four classical scheduling policies, a causal monitoring and degradation detection pipeline, a tri-state verifier with phase-adjusted residuals, and a comprehensive incident ledger recording every decision with its evidence, cost, and outcome.

The evaluation on 210 held-out test traces demonstrates that both the waiting-time rule and the Q-learning controller reduce mean waiting time by approximately 8.5 units over the FCFS baseline, with paired bootstrap confidence intervals excluding zero. However, neither approaches the fixed SJF oracle, and the Q-learning arm is statistically indistinguishable from the hand-written rule. Every accepted intervention was genuinely helpful (65 of 65), but the verifier certifies only 8% of beneficial interventions due to the limited statistical power of a single observation window.

The major conclusion is that safe adaptation is achievable: the system can detect degradation, apply recovery actions, and avoid harmful interventions through verification and rollback. However, the project does not demonstrate a reinforcement-learning advantage over rule-based methods. The Q-learning agent is undertrained, and its limited state-action coverage prevents it from learning a meaningfully different policy from the hand-written rule.

**Future scope (all unimplemented):**

1. Broader workload benchmarks: evaluation against real workload traces from production systems, if available, to test external validity.

2. Additional baselines: comparison against CFS, EEVDF, and other kernel scheduling policies in a simulation environment that models their specific mechanisms.

3. More extensive repeated experiments: additional training seeds, longer training budgets, and larger test sets to improve the statistical power of the Q-learning evaluation.

4. Improved adaptation-state design: reducing the state space to match the event supply, or using function approximation (deep Q-networks) to generalise across unseen states.

5. Real-time integration studies: porting the detection, diagnosis, and verification mechanisms to a real kernel scheduling context, potentially as a loadable module or eBPF program.

6. Multi-objective optimisation: extending the reward function to balance waiting time with fairness, energy consumption, or tail latency.

## 20. References


Brunke, L., Churchill, E., & Amato, N. (2022). Safe learning in robotics: From learning-based control to safe reinforcement learning. Annual Review of Control, Robotics, and Autonomous Systems, *5*, 439-468. https://doi.org/10.1146/annurev-control-042921-103939

Chen, Y., Chen, S., Li, K. C., Liang, W., & Li, Z. (2023). DRJOA: Intelligent resource management optimization through deep reinforcement learning approach in edge computing. Cluster Computing, *26*, 2897-2911. https://doi.org/10.1007/s10586-022-03768-z

Fu, Y., Shi, R., Wang, H., Chen, S., & Cheng, Y. (2024). ALPS: An adaptive learning, priority OS scheduler for serverless functions. In Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24), pp. 19-36. https://www.usenix.org/conference/atc24/presentation/fu

Gu, S., Yang, L., Du, Y., Chen, G., Walter, F., Wang, J., & Knoll, A. (2024). A review of safe reinforcement learning: Methods, theories and applications. IEEE Transactions on Pattern Analysis and Machine Intelligence, 46(12), 11216-11235. https://doi.org/10.1109/TPAMI.2024.3457538

Hafaiedh, I., & Slimane, M. (2022). A distributed formal-based model for self-healing behaviors in autonomous systems: From failure detection to self-recovery. The Journal of Supercomputing, *78*, 18725-18753. https://doi.org/10.1007/s11227-022-04614-0

Hao, S., Li, Y., & Wei, H. (2023). State-wise safe reinforcement learning: A survey. In Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI 2023), 6848-6856. https://doi.org/10.24963/ijcai.2023/763

Hayat, A., Khalid, Y. N., Rathore, M. S., & Nadir, M. N. (2023). A machine learning-based resource-efficient task scheduler for heterogeneous computer systems. The Journal of Supercomputing, 79(14), 15700-15728. https://doi.org/10.1007/s11227-023-05266-4

Ismail, A. A., Khalifa, N. E., & El-Khoribi, R. A. (2025). A survey on resource scheduling approaches in multi-access edge computing environment: A deep reinforcement learning study. Cluster Computing, *28*, 184. https://doi.org/10.1007/s10586-024-04893-7

Kang, F., Cui, L., Ye, Z., & Zhou, Y. (2024). Reliability analysis for systems with self-healing mechanism in degradation-shock dependence processes with changing degradation rate. Reliability Engineering & System Safety, *245*, 109671. https://doi.org/10.1016/j.ress.2023.109671

Karimi-Mamaghan, M., Mohammadi, M., Pasdeloup, B., & Meyer, P. (2022). Learning to select operators in meta-heuristics: An integration of Q-learning into the iterated greedy algorithm for the permutation flowshop scheduling problem. European Journal of Operational Research, 304(3), 1153-1171. https://doi.org/10.1016/j.ejor.2022.03.054

Kaveri, P. R., & Lahande, P. (2023). Reinforcement learning to improve resource scheduling and load balancing in cloud computing. SN Computer Science, *4*, 188. https://doi.org/10.1007/s42979-022-01609-9

Kolker-Hicks, E., Zhang, D., & Dai, D. (2023). A reinforcement learning based backfilling strategy for HPC batch jobs. In SC-W '23: Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis (pp. 1316-1323). https://doi.org/10.1145/3624062.3624201

Kuang, C., Duan, M., Lv, T., Wu, Y., Ren, X., & Wang, L. (2023). ODRL: Application of reinforcement learning in priority scheduling for running cost optimization. Research Square. https://doi.org/10.21203/rs.3.rs-3323844/v1

Kumar, N. (2023). Self-healing networks: AI-based approaches for fault detection and recovery. Power System Technology, 47(4), 371-386. https://doi.org/10.52783/pst.206

Lin, Z., Li, C., Tian, L., & Zhang, B. (2022). A scheduling algorithm based on reinforcement learning for heterogeneous environments. Applied Soft Computing, *130*, 109707. https://doi.org/10.1016/j.asoc.2022.109707

Meyer, V., da Silva, M. L., Kirchoff, D. F., & De Rose, C. A. F. (2022). IADA: A dynamic interference-aware cloud scheduling architecture for latency-sensitive workloads. Journal of Systems and Software, *194*, 111491. https://doi.org/10.1016/j.jss.2022.111491

McClure, S., Ousterhout, A., Shenker, S., & Ratnasamy, S. (2022). Efficient scheduling policies for microsecond-scale tasks. In Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22). https://www.usenix.org/conference/nsdi22/presentation/mcclure

Paduraru, C., Patilea, C. C., & Iordache, S. (2023). Task scheduling: A reinforcement learning based approach. In Proceedings of the 15th International Conference on Agents and Artificial Intelligence (ICAART 2023), Volume 3, pp. 948-955. https://doi.org/10.5220/0011826100003393

Pekeric, I., Groner, R., Witte, T., & Adigun, J. G. (2023). A systematic review on security and safety of self-adaptive systems. Journal of Systems and Software, *203*, 111716. https://doi.org/10.1016/j.jss.2023.111716

Rauba, P., Seedat, N., Kacprzyk, K., & van der Schaar, M. (2024). Self-healing machine learning: A framework for autonomous adaptation in real-world environments. In Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024). https://proceedings.neurips.cc/paper_files/paper/2024/hash/4a86ec12e94ef1fe306362e7bdcd5894-Abstract-Conference.html

Sabek, I., Ukyab, T. S., & Kraska, T. (2022). LSched: A workload-aware learned query scheduler for analytical database systems. In Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22), pp. 1228-1242. https://doi.org/10.1145/3514221.3526158

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Sung, T., et al. (2023). Decentralized application-level adaptive scheduling for multi-instance DNNs on open mobile devices. In Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC '23), pp. 573-586. https://www.usenix.org/conference/atc23/presentation/sung

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

Wachi, A., Shen, X., & Sui, Y. (2024). A survey of constraint formulations in safe reinforcement learning. In Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024). https://doi.org/10.24963/ijcai.2024/913

Wang, X., et al. (2023). Theoretical validation and hardware implementation of dynamic adaptive scheduling for heterogeneous systems on chip. Journal of Low Power Electronics and Applications, 13(4), 56. https://doi.org/10.3390/jlpea13040056

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8(3-4), 279-292. https://doi.org/10.1007/BF00992698

Wei, H., Liu, X., & Ying, L. (2024). Safe reinforcement learning with instantaneous constraints: The role of aggressive exploration. Proceedings of the AAAI Conference on Artificial Intelligence, 38(19), 21708-21716. https://doi.org/10.1609/aaai.v38i19.30170

Yahmed, A. H., Bouchoucha, R., Braiek, H. B., & Khomh, F. (2023). An intentional forgetting-driven self-healing method for deep reinforcement learning systems. In Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023), pp. 1314-1325. https://doi.org/10.1109/ASE56229.2023.00121

Yang, G., et al. (2024). Advancements in Q-learning meta-heuristic optimization algorithms: A survey. WIREs Data Mining and Knowledge Discovery, 14(5), e1548.

https://doi.org/10.1002/widm.1548

Zhou, G., Tian, W., Buyya, R., Xue, R., & Song, L. (2024). Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions. Artificial Intelligence Review, *57*, 124. https://doi.org/10.1007/s10462-024-10756-9

Zhou, T., & Lin, M. (2023). CPU frequency scheduling of real-time applications on embedded devices with temporal encoding-based deep reinforcement learning. arXiv preprint arXiv:2309.03779.

Zhu, Z., et al. (2023). Intelligent scheduling based on reinforcement learning approaches: Applying advanced Q-learning and SARSA reinforcement learning models for the optimisation of job shop scheduling problems. Electronics, 12(23), 4752. https://doi.org/10.3390/electronics12234752

Ziaei, F., & Ranjbar, M. (2023). A reinforcement learning algorithm for scheduling parallel processors with identical speedup functions. Machine Learning with Applications, *14*, 100485. https://doi.org/10.1016/j.mlwa.2023.100485

## 21. Complete Source Code and Supporting Documents

The complete source code and supporting documentation are maintained in the project repository at https://github.com/lostpilot404/OS-Project on branch arena/c2630615-os-project.

**Repository structure:**

**Installation and setup:**

1. Clone the repository: git clone https://github.com/lostpilot404/OS-Project 2. Create virtual environment: python -m venv .venv && source .venv/bin/activate 3. Install dependencies: pip install -r requirements.lock 4. Verify: pytest tests/ 5. Run demo: python main.py demo 6. Launch dashboard: python tools/serve_dashboard.py

## 22. Final Working Demonstration of the Project

**Environment setup:**

1. Ensure Python 3.11 is installed. 2. Create and activate virtual environment. 3. Install requirements: pip install -r requirements-dashboard.txt 4. Verify tests pass: pytest tests/ -q

**Launch commands:**

# CLI demo (text output) python main.py demo # Interactive web dashboard python tools/serve_dashboard.py # Open browser to http://127.0.0.1:8000/

**Demonstration script for live assessment:**

**Step 1: Overview page:** Show the dashboard overview with scenario counts and study summary. Explain that the system monitors scheduling performance and applies recovery when degradation is detected.

**Step 2: Teaching example (3-process):** Navigate to Simulation Studio > Teaching Example. Run the 3-process example with FCFS and SJF. Show the Gantt chart and explain how waiting times differ between policies.

**Step 3: Accepted recovery (seed 13):** Select the 'Accepted Recovery' scenario. Run the simulation. Show the incident with verdict 'accepted' — the system detected degradation, applied a recovery action (switched to SJF), verified the improvement, and kept the change.

**Step 4: Inconclusive rollback (seed 3):** Select the 'Inconclusive Recovery' scenario. Run the simulation. Show the incident with verdict 'inconclusive' — the system detected degradation, applied a recovery, but could not verify improvement, so it rolled back without counting it as a failure.

**Step 5: Policy comparison:** Navigate to the Policy Comparison page. Run a comparison across all four policies on the same workload. Show the bar chart of mean waiting times and explain the differences.

**Step 6: Research results:** Navigate to the Research Results page. Show the committed study summary: held-out comparisons, counterfactual audit, and verification behaviour. Explain that every number is traceable to a CSV file.

**Step 7: Custom workload:** Navigate to Workload Explorer. Create a custom workload with specified arrival times, bursts, and priorities. Run it under different policies. Export as JSON.

**Step 8: Explain the self-healing loop:** Walk through the architecture diagram (Figure 2). Explain: generate → simulate → monitor → detect → diagnose → act → verify → accept/rollback/fallback → record. Emphasize the causal boundary: the controller never sees future arrivals or the finished schedule.

**Step 9: Explain verification:** Show the tri-state verifier (Figure 6). Explain: accepted (keep), rejected (rollback + failure count), inconclusive (rollback, no penalty). Emphasize that the system never keeps a change it cannot verify.

**Step 10: Negative results:** Explain that the Q-learning controller is not better than the rule, the verifier certifies only 8% of helpful interventions, and the fairness rule is genuinely harmful. These are honest findings, not failures of the demonstration.

**Troubleshooting:**

- If the dashboard fails to start, ensure port 8000 is not in use. - If the backend shows errors, run pytest tests/ to check for regressions. - The demo command does NOT train Q-learning; it runs two small deterministic fixtures. - The research results page reads committed CSV files; they are read-only. - JSON export creates a file in the browser's download directory.

**Known limitations for the demonstration:**

- The system is a simulation, not a real kernel scheduler. - All workloads are synthetic; no real-world traces are used. - The Q-learning advantage is not demonstrated; the system is indistinguishable from the rule. - The verifier is a safety filter, not a benefit estimator.

flowchart LR A["Workload<br/>Families<br/>(7 families)"] --> B["Train / Valid /<br/>Test Split<br/>(seeds 7101-7105,<br/>8201, 23819)"] B --> C["Arms<br/>(10 arms:<br/>fixed, rule,<br/>Q-learning)"] C --> D["Evaluation<br/>(210 traces/arm,<br/>bootstrap 1000)"] D --> E["Counterfactual<br/>Audit &<br/>Report"]

flowchart TB subgraph Workload Generation WG["Workload Generator<br/>(workload/)"] --> WF["Workload Fingerprint<br/>(fingerprint.py)"] WF --> DI["Drift Injector<br/>(drift.py)"] DI --> BE["Burst Estimator<br/>(burst_estimator.py)"] end subgraph Scheduling WF --> RS["Discrete-Event Scheduler<br/>(scheduler/runtime.py)"] RS --> RO["Runtime Observation<br/>(RuntimeObservation)"] DI --> RS RO --> PC["Policy Classes<br/>(FCFS/SJF/RR/Pri)"] BE --> RS end subgraph Self-Healing Controller RS --> CM["Causal Monitor<br/>(monitor.py)"] CM --> DD["Degradation Detector<br/>(detector.py)"] DD --> DX["Diagnosis<br/>(diagnosis.py)"] DX --> RP["Recovery Policy<br/>(policies.py)"] end subgraph Verification and Outcome RP --> VW["Verification Window<br/>(verifier.py)"] VW --> AR["Accept / Reject / Inconclusive"] AR --> RB["Rollback / Fallback<br/>(actions.py)"] RB --> IL["Incident Ledger"] end subgraph Evaluation VW --> MT["Metrics<br/>(evaluation/metrics.py)"] AR --> RL["Reliability<br/>(evaluation/reliability.py)"] RB --> BP["Paired Comparison<br/>(evaluation/paired.py)"] end RB -.->|"Feedback (action applied to scheduler)"| RS

flowchart TD GW["Generate workload"] --> SM["Simulate (dispatch slice)"] SM --> MN["Monitor (EWMA reference)"] MN --> DD{"Degradation detected?"} DD -- "no: continue simulating" --> SM DD -- "yes" --> DC["Diagnose cause"] DC --> SA["Select

recovery action"] SA --> AA["Apply action to scheduler"] AA --> VW["Verification window"] VW -->|"good"| AC["Accepted (keep change)"] VW -->|"bad"| RJ["Rejected (rollback + failure)"] VW -->|"unclear"| IN["Inconclusive (rollback, no penalty)"] AC --> IL["Incident Ledger"] RJ --> IL IN --> IL AC --> FQ{"2 consecutive failures?"} FQ -- "yes" --> SF["Safe fallback"] FQ -- "no" --> SM

flowchart TB FS["Family + Seed selection"] --> WG["WorkloadGenerator.generate()"] WG --> WO["Workload object (fingerprinted)"] DS["Optional: drift spec (drift.py)"] --> GD["generate_with_drift()"] GD --> DW["Drifted Workload (phase shift)"] WO --> RS["RuntimeSimulator.run(workload, ctrl)"] DW --> RS RS --> EL["Event loop: dispatch, switch, complete"] EL --> SR["ScheduleResult (per-process outcomes)"]

flowchart TD CN["Completion notice from scheduler"] --> RW["Update rolling window (EWMA ref + band)"] RW --> TH{"window_mean > ref + k * band ?"} TH -- "yes" --> HY["Hysteresis check (h consecutive)"] TH -- "no" --> SU["Suppress (reason logged)"] HY --> GA{"Gates pass? (cooldown, load, work)"} GA -- "yes" --> RE["Build ready evidence"] GA -- "no" --> SU RE --> DG["Diagnose: primary cause"] DG --> DI["Declare incident"]

flowchart TD AA["Action applied; window opened"] --> CC["Collect completions over verify_decisions"] CC --> PC["Compute phase-adjusted closure residual"] PC --> AB{"residual >= accept boundary (+0.30)?"} AB -- "yes" --> AC["ACCEPTED (keep, learn)"] AB -- "no" --> RB{"residual <= reject boundary (-0.30)?"} RB -- "yes" --> RJ["REJECTED (rollback, fail +1)"] RB -- "no" --> IC["INCONCLUSIVE (rollback, no penalty)"]

flowchart LR UI["Browser UI (app.js)"] --> HR["HTTP Request POST /api/simulations"] HR --> FR["FastAPI Route (api.py)"] FR --> SL["Service Layer (service.py)"] SL --> SE["Simulation Engine (demo.run_fixture)"] SE --> MV["Model Validator (model.py)"] MV --> JR["JSON Response (workload, metrics, incidents)"] JR --> CH["Charts (inline SVG)"] CH --> RU["Render in UI (index.html)"] JR -.->|"JSON + SVG"| UI

■■■

| Reference | Topic | Venue | Key Relevance |
|---|---|---|---|
| Kumar (2023) | Self-healing networks | Power Sys Tech | AI-based fault detection |
| Zhou & Lin (2023) | CPU freq scheduling DRL | arXiv 2309.03779 | Temporal encoding DVFS |
| Hayat et al. (2023) | ML task scheduler | J Supercomputing | Learned execution predictors |
| Ziaei & Ranjbar (2023) | RL parallel processor sched | ML with Applications | Identical speedup functions |
| Fu et al. (2024) | ALPS adaptive priority sched | USENIX ATC 2024 | Kernel-level adaptive priority |
| Sung et al. (2023) | Decentralized adaptive sched | USENIX ATC 2023 | Nash equilibrium scheduling |
| Karimi-Mamaghan et al. (2022) | Q-learningin metaheuristics | Eur J Oper Res | Learned operator selection |
| Zhu et al. (2023) | Q-learning/SARSA for JSSP | Electronics (MDPI) | Tabular RL for job shops |
| Yang et al. (2024) | Q-learning metaheuristic survey | WIREs Data Mining | Advancements survey |
| Meyer et al. (2022) | IADA dynamic scheduling | J Sys Software | ML classification + scheduling |
| Kang et al. (2024) | Self-healing reliability | Reliability Eng Sys | Degradation-shock models |

| Reference | Topic | Venue | Key Relevance |
|---|---|---|---|
| Pekeric et al. (2023) | Safety of self-adaptive sys | J Sys Software | Systematic review |
| Wachi et al. (2024) | Constraint formulations safe RL | IJCAI 2024 | Constraint taxonomy |
| Wei et al. (2024) | Safe RL instantaneous constraints | AAAI 2024 | LSVI-AE algorithm |
| Yahmed et al. (2023) | Self-healing for DRL systems | ASE 2023 | Intentional forgetting method |
| Rauba et al. (2024) | Self-healing ML framework | NeurIPS 2024 | Autonomous diagnosis + correction |
| Chen et al. (2023) | Energy-aware HPC scheduling | Energies (MDPI) | Survey of energy-aware methods |
| Kuang et al. (2023) | ODRL priority scheduling | Research Square | DRL for datacenter cost |
| Ismail et al. (2025) | DRL MEC resource scheduling | Cluster Computing | Survey of edge scheduling |
| Brunke et al. (2022) | Safe RL real-world robotics | Ann Rev Control | Sim-to-real safety transfer |
| Wang et al. (2023) | Dynamic adaptive scheduling | JLPEA | Runtime classifier for SoC |

| Module (file) | Key Class/Function | Responsibility |
|---|---|---|
| workload/generator.py | WorkloadGenerator | Generate reproducible workload instances from (family, seed) pairs |
| workload/models.py | Process, Workload, ScheduleResult | Data structures for processes, workloads, and schedule outcomes |
| workload/fingerprint.py | ContentFingerprint | SHA-256 content fingerprint for workload identity verification |
| workload/drift.py | DriftSpec, apply_drift | Inject mid-trace workload phase shifts |
| scheduler/runtime.py | RuntimeSimulator | Discrete-event scheduling engine with causal observation boundary |
| scheduler/fcfs.py, sjf.py, round_robin.py, priority.py | Policy classes | Four classical scheduling implementations |
| healing/config.py | HealingConfig | All thresholds, gates, costs, and bounds for the healing loop |
| healing/monitor.py | CausalMonitor | Rolling completion window, EWMA reference and dispersion band |
| healing/detector.py | DegradationDetector | Detect degradation events with hysteresis, cooldown, and gates |
| healing/diagnosis.py | build_ready_evidence, diagnose | Map ready-set evidence to primary degradation cause |
| healing/actions.py | apply_healing_action | 8-action recovery space and configuration application |
| healing/policies.py | QLearningHealingPolicy, MappedRulePolicy | Recovery policy selection (learned and rule-based) |

| Module (file) | Key Class/Function | Responsibility |
|---|---|---|
| healing/verifier.py | verify | Tri-state verification with phase-adjusted residuals |
| healing/controller.py | SelfHealingController | State machine: monitor, detect, diagnose, act, verify, rollback |
| healing/reward.py | controller_reward | Reward computation for the Q-learning agent |
| rl/q_learning.py | QLearningAgent | Tabular Q-learning with epsilon-greedy exploration |
| rl/state.py, rl/adaptive.py | State encoder, selector | Offline workload-aware policy selector |
| evaluation/metrics.py | compute_metrics | Compute waiting, turnaround, response times, utilization, throughput, switches |
| evaluation/paired.py | paired_bootstrap | Family-stratified crossed bootstrap comparisons |
| evaluation/reliability.py | compute_reliability_metrics | P95, starvation, fairness, recovery delay, false-intervention rates |
| dashboard/api.py | FastAPI routes | HTTP endpoints for the interactive web console |
| dashboard/service.py | Simulation service | Adapter connecting API to engine modules |
| dashboard/web/ | index.html, app.js, styles.css | Browser-based interactive console |

| Family | Processes | Burst Distribution | Arrival Pattern | Priority Pattern |
|---|---|---|---|---|
| short_jobs | 15 | bimodal (80% short) | uniform | uniform |
| long_jobs | 15 | bimodal (20% short) | uniform | uniform |

| Family | Processes | Burst Distribution | Arrival Pattern | Priority Pattern |
|---|---|---|---|---|
| mixed | 15 | uniform [1,50] | uniform | uniform |
| cpu_bursty | 15 | uniform [20,50] | uniform (dense) | uniform |
| priority_skewed | 15 | uniform [1,50] | uniform | 70% high priority |
| staggered_interactive | 15 | staggered (1 long + 14 short) | staggered (gap 4-6) | uniform (1) |
| poisson_arrivals | 15 | uniform [1,20] | poisson (rate 0.5) | uniform |

| Parameter | Value |
|---|---|
| Round Robin quantum | 4 time units |
| Context-switch cost | 1 time unit (in healing study) |
| Training episodes per seed | 1200 |
| Training seeds | 7101, 7102, 7103, 7104, 7105 |
| Validation seed | 8201 |
| Test seed | 23819 |
| Bootstrap replicates | 1000 (seed 104729) |
| Monitor window | 8 completions |
| Detection threshold | 1.25x band above reference |
| Hysteresis | 2 consecutive evaluations |
| Cooldown | 3 decisions |
| Verification decisions | 5 (min 3 completions, max 14) |
| Accept boundary | +0.30 (phase-adjusted residual) |
| Reject boundary | -0.30 (phase-adjusted residual) |
| Intervention cost | 1% of reference waiting time |

| Parameter | Value |
|---|---|
| Max interventions per trace | 6 |
| Max consecutive failures | 2 |
| Q-learning rate (alpha) | 0.1 |
| Discount factor (gamma) | 0.9 |
| Epsilon schedule | 1.0 -> 0.05 (decay 0.995/episode) |
| State space size | 648 states (healing), 81 states (offline) |
| Action space size | 8 actions (healing), 4 actions (offline) |

| Metric | Formula | Definition | Units | Preference |
|---|---|---|---|---|
| Waiting Time (W_i) | W_i = S_i - A_i | Start time minus arrival time | Time units | Lower is better |
| Mean Waiting Time | W_bar = (1/n) * sum(W_i) | Average per process | Time units | Lower is better |
| Turnaround Time (T_i) | T_i = C_i - A_i | Completion minus arrival | Time units | Lower is better |
| Response Time (R_i) | R_i = F_i - A_i | First execution minus arrival | Time units | Lower is better |
| CPU Utilization | U = 100 * B / E | Busy time / elapsed time * 100 | Percent | Higher is better |
| Throughput | P = n / E | Completed processes / elapsed time | proc/time unit | Higher is better |
| Context Switches | CS = count of running process changes | Number of dispatch changes | Count | Lower is better |

| Arm | Mean Wait | P95 Wait | Mean Resp | Mean TAT | CPU Util% | Throughput | Ctx Switches |
|---|---|---|---|---|---|---|---|
| fixed_sjf | 94.66 | 249.20 | 94.66 | 113.59 | 92.27 | 0.0778 | 14.00 |
| q_recovery_from_sjf | 94.66 | 249.20 | 94.66 | 113.59 | 92.27 | 0.0778 | 14.00 |
| rule_wait_from_sjf | 94.87 | 249.54 | 94.87 | 113.81 | 92.27 | 0.0778 | 14.00 |
| rule_wait | 122.39 | 251.88 | 122.39 | 141.33 | 92.27 | 0.0778 | 14.00 |

| Arm | Mean Wait | P95 Wait | Mean Resp | Mean TAT | CPU Util% | Throughput | Ctx Switches |
|---|---|---|---|---|---|---|---|
| q_recovery (5 seeds) | 122.56 | 251.98 | 122.54 | 141.50 | 92.25 | 0.0778 | 14.03 |
| fixed_priority | 128.32 | 262.75 | 128.32 | 147.25 | 92.27 | 0.0778 | 14.00 |
| detector_only (FCFS) | 130.84 | 258.42 | 130.84 | 149.77 | 92.27 | 0.0778 | 14.00 |
| fixed_fcfs | 130.84 | 258.42 | 130.84 | 149.77 | 92.27 | 0.0778 | 14.00 |

| Arm | Mean Wait | P95 Wait | Mean Resp | Mean TAT | CPU Util% | Throughput | Ctx Switches |
|---|---|---|---|---|---|---|---|
| rule_fairness | 136.78 | 263.72 | 110.40 | 155.72 | 90.38 | 0.0766 | 20.02 |
| fixed_round_robin | 217.05 | 310.41 | 26.50 | 235.99 | 78.25 | 0.0687 | 75.26 |

| Target | Reference | Metric | Difference | 95% CI |
|---|---|---|---|---|
| | | avg_waiting_time | | [, ] |
| | | p95_waiting_time | | [, ] |
| | | max_waiting_time | | [, ] |
| | | avg_turnaround_time | | [, ] |
| | | avg_response_time | | [, ] |
| | | cpu_utilization | | [, ] |
| | | throughput | | [, ] |
| | | context_switches | | [, ] |

| Target | Reference | Metric | Difference | 95% CI |
|---|---|---|---|---|
| | | avg_waiting_time | | [, ] |

| Arm | Audited | Mean Gain | Helped | False Recoveries | False Rate |
|---|---|---|---|---|---|
| rule_fairness | 150 | -130.8 | 35 | 115 | 76.7% |
| rule_wait | 141 | 188.8 | 133 | 8 | 5.7% |
| rule_wait_from_sjf | 4 | -169.2 | 0 | 4 | 100.0% |
| q_recovery | 678 | 192.1 | 644 | 34 | 5.0% |

| Arm | Interventions | Accepted | Rejected | Inconclusive | Confirmed Failures |
|---|---|---|---|---|---|
| rule_fairness | 150 | 2 (1.3%) | 31 (20.7%) | 117 (78.0%) | 31 |
| rule_wait | 141 | 12 (8.5%) | 0 | 129 (91.5%) | 0 |

| Arm | Interventions | Accepted | Rejected | Inconclusive | Confirmed Failures |
|---|---|---|---|---|---|
| q_recovery | 678 | 51 (7.5%) | 2 (0.3%) | 625 (92.2%) | 2 |
| rule_wait_from_sjf | 4 | 0 | 1 (25%) | 3 (75%) | 1 |

| Method | Adaptation | Verification | Setting | Strengths | Limitations |
|---|---|---|---|---|---|
| This project (rule_wait) | Rule-based diagnosis->action | Tri-state verifier + rollback | Simulation, 7 families | Causal boundary, audit, transparent thresholds | Simulation only, no RL advantage |
| This project (q_recovery) | Tabular Q-learning | Tri-state verifier + rollback | Simulation, 7 families | Learned recovery, same safety guarantees | Undertrained, indistinguishable from rule |
| Zhou et al. (2024) | DRL methods survey | None (survey) | Various | Comprehensive taxonomy | No implementation |
| Sabek et al. (2022) | RL query scheduler | Latency optimization | Database benchmark | Workload-aware learned scheduling | Database domain, no verification |
| Kolker-Hicks (2023) | RL backfilling | None | HPC simulation | Tabular Q-learning for scheduling | No safety mechanism |

| Method | Adaptation | Verification | Setting | Strengths | Limitations |
|---|---|---|---|---|---|
| Gu et al. (2024) | Safe RL survey | Constraint methods | Various | Comprehensive safety taxonomy | No scheduling focus |
| Hafaiedh (2022) | Formal self-healing | Formal verification | Autonomous systems | Formal specification of recovery | Not scheduling-specific |
| Fu et al. (2024) | ALPS kernel scheduler | Learned priority | Linux kernel | Real OS integration | No explicit verification |

| Path | Description |
|---|---|
| config.py | Configuration dataclasses for schedulers, workloads, Q-learning, training |
| errors.py | Custom exception types |
| main.py | CLI entry point with subcommands |
| scheduler/ | Discrete-event simulator and four policy implementations |
| workload/ | Workload generation, models, fingerprinting, drift injection |
| healing/ | Self-healing controller: monitor, detector, diagnosis, actions, policies, verifier, reward |
| rl/ | Q-learning agent, state encoder, adaptive selector, runtime controller |
| evaluation/ | Metrics, paired bootstrap, counterfactual, reliability |
| experiments/ | Study runners, training, healing experiment, drift experiment |
| visualization/ | Matplotlib figure generation from study artifacts |
| dashboard/ | FastAPI server, browser console, teaching examples, research viewer |

| Path | Description |
|---|---|
| dashboard/web/ | index.html, app.js, styles.css — interactive web console |
| tests/ | pytest test suite (500+ tests) |
| tools/ | Build scripts for reports, dashboard, teacher HTML |
| docs/ | Architecture, methodology, limitations, audit, demo guides |
| results/healing/ | Frozen study artifacts (CSV, JSON, incident ledgers) |
| results/study/ | Offline policy-selection study artifacts |
| results/runtime/ | Runtime experiment artifacts |
| figures/ | Generated figures from study artifacts |
| requirements.txt | Runtime dependencies |
| requirements.lock | Pinned dependency versions |